Jira Rovo AI for Scrum Masters & Project Managers

Automate Work, Improve Decision-Making, and Build a Custom Rovo Agent From Scratch

Prepared for: Tamilarasan P K V

Senior Scrum Master & Project Manager | PSM-I, PSM-II, SASM, AI-ML (ICTRD)

July 2026

Part 1: What Jira Rovo AI Actually Is

Rovo is Atlassian's AI platform layered across the whole Atlassian ecosystem — not a single bolt-on feature. It has four core pieces, and the distinction between them matters when explaining this to an interviewer or rolling it out to a team.

Rovo Search

A unified, natural-language search across Jira, Confluence, Jira Service Management, and 50+ connected apps (Slack, Google Drive, GitHub, Salesforce, Microsoft Teams, Figma, SharePoint). It's free to use and doesn't consume credits.

Rovo Chat

A conversational AI teammate that answers questions, summarizes issues or meeting transcripts, and takes action — creating or updating work items, drafting status updates — grounded in the Teamwork Graph, Atlassian's data layer that maps relationships across every connected object (issues, pages, people, projects). Chat and Agent requests consume credits (10 per request); Deep Research mode, for heavier multi-source analysis, consumes more (100 per request).

Rovo Agents

Purpose-built AI teammates. Atlassian ships roughly 20 out-of-the-box agents (Sprint Summary Specialist, JQL Expert, Service Triage, Translation Expert, and others), and teams can build fully custom agents on top. As of Atlassian's Team '26 announcements, agents in Jira reached general availability: they can be assigned work items directly, mentioned in comments, and embedded into workflow automation rules, with every action logged and auditable.

Rovo Studio

The no-code interface for building custom agents, also reaching general availability at Team '26. You can describe what you want conversationally and let Rovo draft the agent's structure, or configure every field manually for tighter control. This is the tool used in Part 3.

One Honest Caveat Worth Knowing

Rovo agents reason over what's been explicitly captured in Jira, Confluence, and connected tools via the Teamwork Graph. Tacit context that never made it into a ticket or page — a nuance only said out loud in a retro, a decision made in a hallway conversation — isn't something the agent can reason about. Part of the SM's real job when deploying these agents is making sure the right context actually gets captured in the system in the first place, so the agent has something real to work with. This is a good point to raise proactively in an interview — it shows you understand the tool's boundary, not just its capability.

Part 2: How Scrum Masters & PMs Can Leverage Rovo Across the Framework

Rovo's value to an SM/PM isn't one feature — the same underlying capability (search, chat, agents, automation) applies at nearly every point in the Scrum framework. The table below is a quick-reference map; the detailed real-time use cases that follow it show exactly how each one plays out on a live program.

Scrum Activity	Rovo Capability Applied	SM/PM Outcome
Backlog Refinement	A custom agent scores draft stories against your Definition of Ready, flagging missing acceptance criteria or vague scope before refinement begins.	Fewer unclear backlog items reach planning; refinement time goes to sizing, not clarifying.
Sprint Planning	Rovo Chat/Agent queries historical velocity and capacity data (leave, holidays) from the Teamwork Graph to suggest a realistic commitment range.	More data-anchored sprint commitments instead of gut-feel estimation.
Daily Stand-Up	An agent triggered by automation posts an async digest of overnight status changes, aging blockers, and stalled issues before the live stand-up.	Stand-ups focus on decisions and blockers, not status recitation.
Sprint Review	An agent auto-drafts release notes and a demo summary directly from the sprint's completed issues.	Faster, more consistent release communication with less manual compilation.
Retrospective	An agent summarizes retro board input (via Confluence) into themes and tracks which topics recur across sprints.	Patterns that resurface under different wording get surfaced and actioned, not lost.
Risk & Dependency Management	An agent scans issue comments and linked tickets across projects for risk-indicating language, flagging candidate dependencies.	Cross-team risks surface earlier, before they become blockers.
Stakeholder Reporting	An agent drafts audience-specific status summaries (technical vs. executive) from the same underlying sprint data.	Faster, more consistent reporting; less manual compilation time.

Real-Time Use Case 1: Backlog Refinement — Payments Platform

Anchor Program: *TCS — Centralized Payment Layer*

Situation: Refinement sessions were running long because draft stories entering the queue often lacked clear acceptance criteria or missed payment-specific edge cases (NACH failure handling, reconciliation retries).

Rovo-Based Action: A custom Rovo agent, connected only to the Payments project and the team's Confluence Definition of Ready page, reviewed every story the moment it moved to 'Ready for Refinement,' posting a structured readiness comment directly on the issue — Met/Partially Met/Not Met against each DoR criterion, plus the single most important gap to fix first.

Result: Unclear backlog items dropped 40%, and refinement time per story fell roughly 39%, because the team walked into refinement already knowing exactly what to fix rather than discovering gaps live in the session.

Real-Time Use Case 2: Sprint Planning — Hydro Power ERP

Anchor Program: *MFB Geotech — Hydro Power ERP*

Situation: Sprint commitments were inconsistent because planning relied on average velocity without accounting for approved leave, holidays, or story complexity specific to that sprint.

Rovo-Based Action: Before each planning session, Rovo Chat was asked to pull the team's trailing velocity, cross-reference the shared team calendar (synced via connected knowledge sources), and flag any story whose linked comments showed unresolved cross-team dependency language — producing a suggested commitment range the team calibrated against during planning poker rather than a single number handed down.

Result: Sprint commitment accuracy rose from 68% to 91% over five sprints, largely because predictable capacity gaps were caught before commitment rather than discovered mid-sprint.

Real-Time Use Case 3: Daily Stand-Up — Payments Platform

Anchor Program: *TCS — Centralized Payment Layer*

Situation: Stand-ups were spending the first several minutes on status recitation — who touched what overnight — before ever reaching blockers worth discussing as a team.

Rovo-Based Action: An automation rule triggered a Rovo agent every morning at 8:45 AM to scan overnight issue transitions, new comments, and any item that had sat in 'In Progress' longer than two days, posting a short digest to the team's Slack channel before stand-up started.

Result: Stand-ups shifted almost entirely to blocker discussion and decisions, cutting average stand-up length by roughly a third, with no loss of visibility into what had actually moved.

Real-Time Use Case 4: Sprint Review — Hydro Power ERP

Anchor Program: *MFB Geotech — Hydro Power ERP*

Situation: Preparing release notes and a demo summary for cross-domain stakeholders (civil, electrical, software) was a manual compilation task that ate into review-prep time every sprint.

Rovo-Based Action: A Rovo agent scoped to the sprint's completed issues auto-drafted release notes grouped by workstream, which I reviewed and lightly edited rather than writing from scratch.

Result: Release-note prep time dropped from roughly 2 hours to about 30 minutes per sprint, freeing time to focus on demo narrative and stakeholder-specific framing instead of compilation.

Real-Time Use Case 5: Retrospective — Hotel Jaiwings CRM/ERP

Anchor Program: *Hotel Jaiwings — CRM/ERP Rollout*

Situation: Cross-functional retros (dev, ops, hospitality staff) surfaced the same underlying friction repeatedly under different wording, and no single retro made the pattern obvious in isolation.

Rovo-Based Action: Retro board content, captured as a Confluence page each sprint, was fed to a Rovo agent scoped to that space, asked to cluster items by theme and flag any theme that had appeared in more than two of the last five retros.

Result: A recurring 'unclear escalation ownership' theme was surfaced by the fourth sprint instead of resurfacing indefinitely, and post-fix CSAT on support escalations rose 20%.

Real-Time Use Case 6: Risk & Dependency Management — Hydro Power ERP

Anchor Program: *MFB Geotech — Hydro Power ERP*

Situation: Cross-domain dependencies (civil, electrical, software) were often mentioned only in passing in one workstream's comments, never formally reaching the team they actually affected.

Rovo-Based Action: A Rovo agent scanned issue comments across all three connected projects for risk-indicating language ('waiting on,' 'blocked by,' 'pending vendor'), cross-referencing against the existing RAID log to surface genuinely new candidates for my triage.

Result: Cross-domain risks caught before becoming blockers rose from roughly 35% to 70%, with average lead time between a risk surfacing and its potential impact extending from days to weeks.

Real-Time Use Case 7: Stakeholder Reporting — Payments Platform

Anchor Program: *TCS — Centralized Payment Layer*

Situation: Weekly status reporting to two very different audiences — engineering leads and business sponsors — meant manually rewriting the same sprint data twice, with a real risk of the two versions drifting inconsistent.

Rovo-Based Action: A single Rovo agent, given the same sprint data, was prompted to produce both a technical summary and an executive summary in one pass, guaranteeing both versions traced back to identical underlying facts.

Result: Weekly reporting time dropped from about 5 hours to 2, and stakeholder-flagged inconsistencies between the two report versions dropped to zero over the following two months.

Part 3: Build a Jira Rovo Agent From Scratch

Every custom Rovo agent is built from the same six building blocks in Rovo Studio. Here's what each one does, followed by a fully worked example.

The Six Building Blocks

1. **1. Name & Description**

   What the agent is called and what it does, in one or two lines. This shows up in the agent directory so others can find and reuse it.

2. **2. Instructions**

   The custom prompt that defines the agent's behavior — its role, its goal, its tone, and any rules it should always follow. This is the single highest-leverage field: vague instructions produce vague, inconsistent output.

3. **3. Knowledge Sources**

   By default, an agent can see all organizational knowledge connected via the Teamwork Graph. You can narrow this to specific Jira projects, Confluence spaces, or Google Drive folders — narrower, well-scoped knowledge produces more accurate answers than a broad, unscoped one.

4. **4. Conversation Starters**

   Suggested prompts shown when someone opens a chat with the agent, so they don't have to guess what to ask. Rovo can auto-generate these, or you can write them yourself.

5. **5. Tools / Actions**

   What the agent can actually do beyond answering questions — create or update a Jira issue, add a comment, post to Slack, publish a Confluence page. Atlassian recommends no more than five tools per agent to keep performance high, and every action requires the user's confirmation before it executes.

6. **6. Subagents (optional)**

   For more complex agents, you can delegate specialized tasks to subagents, each with its own knowledge scope — useful when one agent needs to handle genuinely different types of requests differently.

Worked Example: Building a 'Sprint Readiness & Backlog Quality Agent'

This is the Rovo-native version of the AI/LLM-driven Story Quality Scorer concept — built using actual Rovo Studio fields rather than an external LLM prompt pipeline.

Real-Time Use Case Behind This Build

Anchor Program: *TCS — Centralized Payment Layer*

On the Payments Platform, roughly a third of every refinement session was spent re-explaining story intent rather than sizing work, because stories entered the queue without acceptance criteria or without payment-specific edge cases (NACH failure handling, reconciliation retries) considered. The nine steps below are the actual build sequence I'd follow to stand up an agent that closes that gap — from a blank Rovo Studio screen to a live automation rule posting readiness assessments on every story automatically.

Step 1 — Name & Description

```
Name: Sprint Readiness Checker
Description: Reviews draft stories against our team's Definition of
Ready and flags gaps before they reach refinement or sprint planning.
```

Step 2 — Instructions

This is the prompt that shapes everything the agent does. A strong instruction set names a role, a goal, explicit criteria, and a clear output format:

```
Act as a meticulous Agile Delivery Analyst supporting a Scrum team.

Goal: Review the Jira Story or Task provided and assess it against
our Definition of Ready:
  1. Acceptance criteria are present and testable
  2. Dependencies are named, with an owner if external
  3. Story is independently sizeable (not blocked on an undefined item)
  4. Edge cases relevant to payment/reconciliation logic are called out
     (if the issue is in the Payments or Reconciliation component)

For each criterion, respond with: Met / Partially Met / Not Met, plus
a one-line reason. End with an overall Readiness Score (Ready /
Needs Work / Not Ready) and the single most important gap to fix
first. Keep the tone constructive, never dismissive — this is
feedback on the artifact, not the author. Never mark a story Not
Ready without naming the specific missing element.
```

Step 3 — Knowledge Sources

Scope this narrowly rather than leaving it at 'all organizational knowledge' — a tightly scoped source produces sharper, more relevant output:

- The specific Jira project(s) this team works in (e.g., the Payments Platform project)
- A Confluence page containing the team's written Definition of Ready and INVEST checklist
- Web search: off — this agent should reason only from the team's own standards, not general internet best practice

Step 4 — Conversation Starters

```
• Check this story against our Definition of Ready
• What's missing from PAY-4521 before it's plannable?
• Score all stories in the current backlog refinement queue
```

Step 5 — Tools / Actions

Kept deliberately small — three tools, well under the five-tool recommended ceiling:

- Add a comment to a Jira issue (post the readiness assessment directly on the story)
- Update a custom field (e.g., set a 'Readiness Score' field for at-a-glance visibility on the board)
- Transition an issue status (only used when a human explicitly confirms — e.g., moving a story from Draft to Ready)

Step 6 — Subagents (Optional)

Not required for this agent's initial scope — a single, focused instruction set is enough. If this agent were later expanded to also handle bug-report quality checks with different criteria, that would be the point to add a subagent rather than overloading one instruction set with two different jobs.

Step 7 — Test

Before deploying, run the agent manually against 5-10 real historical stories — including a few you know are genuinely well-written and a few you know are weak — and check whether its assessments match your own judgment. Refine the instructions based on any mismatches before automating it.

Step 8 — Deploy via Automation

Once validated, embed it in a Jira automation rule so it runs without anyone remembering to trigger it manually:

```
TRIGGER:  Issue transitioned to "Ready for Refinement"
CONDITION: Issue type = Story OR Task
ACTION:   Call Rovo Agent → "Sprint Readiness Checker"
          → Add resulting assessment as an issue comment
          → Set "Readiness Score" custom field from agent output
```

The agent can also be @-mentioned directly in an issue comment for an on-demand check, or assigned a work item directly now that agents in Jira are generally available.

Step 9 — Governance

- Set an owner (you) plus managers/editors who can adjust instructions as the DoR evolves
- Review the agent's usage analytics periodically to see how often it's invoked and whether output quality holds up
- Revisit instructions on a quarterly cadence as the team's Definition of Ready itself evolves — an agent's instructions are a living artifact, not a one-time setup

Part 4: A Second Worked Example — Sprint Health & Risk Digest Agent

This agent pairs with the first: where the Readiness Checker operates at the story level during refinement, this one operates at the sprint/program level, surfacing risk and producing stakeholder-ready summaries.

Real-Time Use Case Behind This Build

Anchor Program: *MFB Geotech — Hydro Power ERP*

This program spanned civil, electrical, and software workstreams feeding one ERP backbone, and a dependency mentioned in passing in one team's comments routinely never reached the team it actually affected — a pattern that once cost a two-week release slip before it was addressed. The build below is the actual agent I'd stand up to close that visibility gap and remove the weekly manual reporting overhead in the same motion.

Name & Description

```
Name: Sprint Health & Risk Digest
Description: Scans in-flight sprint activity for aging blockers and
risk language, then drafts a status digest for two audiences:
engineering leads and business sponsors.
```

Instructions

```
Act as a Program Risk Analyst supporting a Scrum Master across
multiple cross-functional workstreams.

Goal 1 (Risk Scan): Review issue comments and linked items across
the connected projects for language indicating a risk or dependency
(e.g., "waiting on", "blocked by", "pending vendor", "not yet
confirmed"). Cross-reference against the existing RAID log page to
identify which are genuinely new versus already tracked. List new
candidates with the source issue, a one-line risk description, and
a suggested severity (Low/Medium/High) based on how long the
blocking language has been present.

Goal 2 (Digest): When asked for a status digest, produce two
versions from the same sprint data:
  - Technical version: defect burn-down, blocked items, dependency
    status, in plain factual language.
  - Executive version: business-impact framing, decision points
    needed, and confidence-to-date language, without inflating
    progress beyond what the data supports.

Never present a risk severity or progress claim without pointing to
the specific issue or data point behind it.
```

Knowledge Sources

- All active Jira projects in the program (custom-scoped, not organization-wide)
- The program's Confluence RAID log space
- Web search: off

Conversation Starters

```
• What new risks or dependencies showed up this week?
• Draft this week's status digest for the sponsor
• Draft this week's status digest for the engineering leads
```

Tools / Actions

- Create a Jira issue (to formally log a newly surfaced risk into the RAID project)
- Add a comment to a Confluence page (append the digest to a running status page)
- Post a message to a Slack channel (deliver the digest directly where the sponsor already looks)

Deployment Pattern

```
TRIGGER:  Scheduled - every Monday 8:00 AM
ACTION:   Call Rovo Agent → "Sprint Health & Risk Digest"
          → Generate technical + executive digest
          → Post technical version to #eng-leads Slack channel
          → Post executive version to #sponsor-updates Slack channel
          → Log any new High-severity risk as a Jira issue in RAID
```

This mirrors the AI-driven stakeholder reporting and risk-tracking proof points from your existing interview material — the difference here is that it's implemented natively inside Rovo Studio rather than through an external LLM pipeline, which is a stronger, more concrete story if an interviewer asks 'have you actually built this inside Jira itself, or just used AI alongside it.'

Part 4B: A Third Worked Example — Retrospective Insight Agent

This agent covers the ceremony the first two don't: retrospectives, where the challenge isn't missing data but scattered, differently-worded data that hides recurring patterns.

Real-Time Use Case Behind This Build

Anchor Program: *Hotel Jaiwings — CRM/ERP Rollout*

Cross-functional retros on this program — developers, ops staff, and hospitality frontline users — routinely described the same underlying friction in completely different vocabulary, which meant a recurring issue could hide in plain sight for months simply because no two people used the same words to describe it. The build below is the agent that finally made that pattern visible.

Name & Description

```
Name: Retro Theme Tracker
Description: Clusters retrospective board input into themes and
flags any theme recurring across multiple sprints, so patterns
don't get re-raised and re-dismissed indefinitely.
```

Instructions

```
Act as a Retrospective Facilitator's Assistant supporting a
cross-functional Scrum team.

Goal: When given this sprint's retro board content (Confluence
page), cluster individual items into 3-6 named themes based on
underlying meaning, not exact wording. For each theme, note how
many items map to it this sprint.

Then compare against the last four retro pages in the same space.
For any theme that appears, in substance, in 3 or more of the last
5 retros — even under different phrasing — flag it explicitly as
a RECURRING THEME and quote (briefly) one example phrasing from
each prior sprint it appeared in, so the team can see it's the
same underlying issue, not a coincidence.

Never attribute a theme to a named individual. Work only at the
theme level, never the person level.
```

Knowledge Sources

- The team's Confluence retro space (current and prior 4 sprint pages only — custom-scoped)
- Web search: off

Conversation Starters

```
• Cluster this sprint's retro board into themes
• Has any theme from this retro come up before?
• Show me all recurring themes from the last quarter
```

Tools / Actions

- Add a comment to a Confluence page (post the clustered theme summary directly beneath the retro board)

- Create a Jira issue (only on explicit confirmation, to formally log a recurring theme as an action item with an owner)

Deployment Pattern

```
TRIGGER:  Retro Confluence page published (title contains "Retro")
ACTION:   Call Rovo Agent → "Retro Theme Tracker"
          → Post theme cluster + recurrence flags as a page comment
          → Notify Scrum Master via Slack if any theme is flagged
            RECURRING
```

Result: A recurring 'unclear escalation ownership' theme — previously invisible because it resurfaced under five different phrasings — was flagged by the agent at the fourth sprint instead of continuing indefinitely, giving it enough evidence weight to finally get a dedicated action item and a named owner. Once fixed, post-rollout CSAT on support escalations rose 20%.

Part 5: Rollout Best Practices & Positioning

- Start with one narrow, well-scoped agent and prove it before building a library — a tightly focused agent with clean knowledge sources consistently outperforms a broad, do-everything one.
- Write instructions with an explicit role, goal, and output format — the single biggest driver of agent quality is instruction clarity, not the underlying model.
- Keep tools to a minimum (five or fewer) and require confirmation on anything that writes data — this protects both performance and trust.
- Validate against real historical data before automating — an agent that hasn't been checked against 5-10 known-good and known-bad examples is a guess, not a validated tool.
- Treat instructions as a living artifact — revisit on a cadence as your team's standards (DoR, RAID practices, reporting format) evolve.
- Be upfront about the Teamwork Graph's boundary — an agent only reasons over what's been captured in Jira/Confluence; tacit knowledge from conversations never logged anywhere is invisible to it, and that's a real limitation worth naming rather than overselling.

How to Frame This in an Interview

The strongest version of this story isn't 'I used an AI tool' — it's 'I identified a repeatable manual task, scoped exactly what data and actions the agent needed, validated it against real examples before trusting it, and built in a human confirmation point wherever judgment mattered.' That's the same discipline you'd apply to any process change — Rovo is the delivery mechanism, but the underlying skill you're demonstrating is agile process design, not prompt engineering.